A functional Python environment for data science

Lino Galiana
Version of November 2, 2026

This chapter introduces the basics of the `Python` environment for *data science*, focusing on the modularity of the language and the use of `Jupyter` *notebooks*. It presents the essentials for setting up a functional Python environment, explains the advantages of IDEs such as `VSCode`, and offers a hands-on introduction to interactive *notebooks*. The chapter also discusses error handling and the importance of ongoing training in `Python`, providing useful resources for staying up to date in this dynamic ecosystem.

⚠️ **Automatically generated PDF.** This document is produced from the course website and may contain formatting issues. For an up-to-date version adapted to your reading, visit the course page: pythonds.linogaliana.fr.

Table of contents

1 Introduction **1**
2 Ingredients for a functional `Python` environment **3**
2.1 `Python` Interpreter 3
2.2 `Python` Environment 4
3 Using notebooks in the context of this course **7**
4 Exercise to Explore Basic Notebook Features **7**
5 How to solve errors? **8**
6 How to continue learning after this course **9**
6.1 Website content 9
6.2 Technical monitoring 10
Informations additionnelles 10

Skills you will acquire in this chapter

- Understand the key aspects of Pythons modularity
- Get familiar with the day-to-day working environment of a *data scientist*, especially the use of notebooks
- Know how to approach and troubleshoot errors effectively
- Recognize the importance of continuous learning in Python and discover some useful resources to support it.

1 Introduction

The richness of *open-source* languages comes from the ability to use *packages* developed by specialists. `Python` is particularly well-endowed in this area. To caricature, it is sometimes said that `Python` is the second-best language for all tasks, which makes it the best language.

Indeed, the malleability of `Python` means that it can be approached in very different ways depending on whether you are a *SysAdmin*, a web developer, or a *data scientist*. It is this latter profile that will interest us here.

However, this richness poses a challenge when starting to learn `Python` or reusing code written by others. Proprietary statistical languages operate in a *top-down* model, where, once a license is acquired, you simply install the software and follow the documentation provided by the developing company to proceed. In *open source* languages, the approach is more *bottom up*: the ecosystem is enriched by contributions and documentation from various sources. Several ways of doing the same thing coexist, and it is part of the work to take the time to choose the best approach.

There are primarily two ways to use `Python` for *data science*[1]:

1. **Local installation**: Install the `Python` software on your machine, configure a suitable environment (it is recommended to use `uv`), and use a development tool, such as `Jupyter` or `VSCode`, to write and execute `Python` code. We will revisit these three levels of abstraction in the next section.
2. **Using an online environment**: Access a preconfigured `Python` environment via your browser, hosted on a remote machine. This machine will execute the code you edit from your browser. This method is particularly recommended for beginners or for those who do not want to deal with system configuration.

The first approach has the pedagogical drawback of requiring many notions—some of them complex—to be introduced before writing a single line of code. It also makes results less reproducible, since it opens the door to personal configurations that can affect the experience. Conversely, the second approach has the benefit of standardizing the experience by offering a single, common setup for starting a course.

This second approach relies on *cloud* services. In the context of this course, as will be explained later, we offer two solutions:

- Using `SSPCloud`[2] is mandatory for ENSAE and ENSAI students, and recommended for everyone else. This is a *cloud* developed by the French government and made available free of charge to students, researchers, and public officials.
- For those unfortunate enough not to have access to `SSPCloud`, it is possible to use `Google Colab`, which provides computing resources. However, the user experience is rather mediocre—*Google* hasn't really improved the service (aside from sprinkling AI everywhere…) since Covid. If you have a personal `VSCode` installation, you will probably find it more pleasant to open the notebooks that way. It is also possible to open the *notebooks* in `Github Codespaces`, but the outcome isn't guaranteed: this option has never really been tested!

The two methods are briefly described below. The local installation method is presented to introduce useful concepts but is not detailed as it can be complex and prone to configuration issues. If you choose this route, be prepared to encounter technical difficulties and sometimes obscure configurations, often requiring tedious searches and trial-and-error fixes to resolve problems that you would probably not have encountered in an online environment.

[1] A third way to use `Python`, still under development, relies on a technology called *WebAssembly*. This approach allows executing `Python` code directly in the browser, offering a local execution experience without requiring complex installation.

[2] Using this infrastructure is mandatory for ENSAE and ENSAI students! Why? First, because it lets you move past the limits of your personal computer and of Colab for the rest of your studies. It also ensures everyone is guaranteed to get the expected results when writing up the lab exercises.

2 Ingredients for a functional `Python` environment

As mentioned earlier, using `Python` locally for data science relies on three main elements:

1. The `Python` interpreter,
2. A virtual environment,
3. An integrated development environment (IDE).

Each of these components plays a complementary role in the development process.

2.1 `Python` Interpreter

This refers to the programming language itself. Installing `Python` on your machine is the essential first step, as it provides the interpreter needed to run your `Python` code. At this stage, it is only the base language and a command-line tool.

> **i** Illustration
>
> If you have access to a command line in an environment where `Python` is available and properly configured (including being added to the `PATH`), you can already run `Python` through the command line[3]
>
> ```
> python --version
> python -c "print(3+3)"
> echo "print(3+3)" > example.py && python example.py && rm example.py
> ```
>
> **Line 3**
>
> An example of creating a file `example.py` from the command line and using it from the command line. Since creating files from the command line is not a realistic working mode, we will explain how to create files using a text editor.

This use from the command line could also be done interactively. If you run the `python` command from your terminal (with no filename after it), you should see a console appear, starting with a message like:

```
$ python

Python 3.13.13 (main, Aug 17 2026, 02:38:26) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
```

which tells you that you have launched the `Python` software in a given version (here `3.13.13`). You can type commands directly into it, for example `3+3`, which should give you a not-too-surprising result.

[3] You do not have direct access to the command line with `Google Colab`, only indirectly through the notebook interface by prefixing commands with `!`. On `VSCode` services provided by `SSPCloud`, the recommended interface for this course, you can access a command line by clicking `☰ > Terminal > New Terminal`. On a personal installation of `VSCode`, this will be at the top in the `Terminal > New Terminal` menu.

2.2 `Python` Environment

The `Python` language is built, like other open-source languages, with a basic core and additional packages. It is these packages that form the rich and dynamic ecosystem of `Python`, making it such a comfortable language.

`Python` is an open-source language, which means that anyone can offer their code for reuse in the form of a package. There are platforms that centralize community packages, with the main one in the `Python` ecosystem being `PyPI`.

An environment refers to the collection of packages available to `Python` for performing specific tasks. `Python` is not installed with all the packages available on `PyPI`, so it is up to you to enrich your environment by installing new packages as needed.

If your `Python` is correctly configured, you can install new packages using `pip install`[4]. We will use many packages in this course, so this command will come up regularly.

Once installed, a script must declare a package before using it (otherwise `Python` won't know where to find specific functions). This is the purpose of the `import` command. For example, `import pandas as pd` allows you to use the `Pandas` package (provided it is already installed).

> **i** Illustration
>
> Here is an illustration of how package management works in `Python`
>
> ```
> python -c "import geopandas as gpd"
> pip install geopandas
> python -c "import geopandas as gpd ; print(gpd.GeoDataFrame)"
> ```
>
> **Line 1**
> This should cause an error if `GeoPandas` is not installed `ModuleNotFoundError: No module named 'geopandas'`
>
> **Line 3**
> The error should be gone

> **!** Important
>
> It is discouraged to use global imports like `from pkg import *`. For example, consider two modules that provide a `sqrt` function:
>
> ```
> # Don't do that please !
> from numpy import *
> from math import *
> sqrt([4, 3])
> ```
>
> Global imports load all functions and variables from the `numpy` and `math` modules into the global namespace, which can lead to naming conflicts.
>
> First, this can lead to unpredictable results, as if the implementations differ, how do you know whether it's the `numpy` or `math` package function that was used? Here, it will be the `math` package function, imported last, which will cause an error: the `math` module only handles integers, not vectors, unlike `numpy`.
>
> Second, it makes the code less readable, as it becomes difficult to know where each function or variable used comes from, complicating maintenance and debugging.

[4] More details on environments are available in the 3rd-year course "Deployment of Data Science Projects", including different types of virtual environments (`conda` or `venv`) and their implications for `Python` computing chains.

It is therefore preferable to import only the necessary functions or use explicit aliases, such as `import numpy as np` and `import math`, to avoid these issues.

2.2.a Development environments and notebooks

Using `Python` via the command line is fundamental in the application world. However, it's impractical to write your code directly in the command line on a daily basis. Fortunately, there are suitable editors known as **IDEs** (*Integrated Development Environments*).

These are software that provide a convenient interface for writing and executing your code. They offer features to simplify code reading and writing: syntax highlighting, autocompletion, debugging, formal code quality diagnostics, etc.

`Jupyter` notebooks[5] offer an interactive interface that allows you to write `Python` code, test it, and see the result below the instruction rather than in a separate console. `Jupyter` notebooks are essential in the fields of data science and education and research because they greatly simplify exploration and experimentation.

They allow you to combine text in `Markdown` format (a lighter markup text format than `HTML` or $ \LaTeX$), `Python` code, and `HTML` code for visualizations and animations in a single document.

[5] `Jupyter` originated from the `IPython` project, an interactive environment for `Python` developed by Fernando Pérez in 2001. In 2014, the project evolved to support other programming languages in addition to Python, leading to the creation of the Jupyter project. The name "Jupyter" is an acronym referring to the three main languages it supports: `Julia`, `Python`, and `R`. `Jupyter` notebooks are crucial in the fields of data science and education and research because they greatly simplify exploration and experimentation.

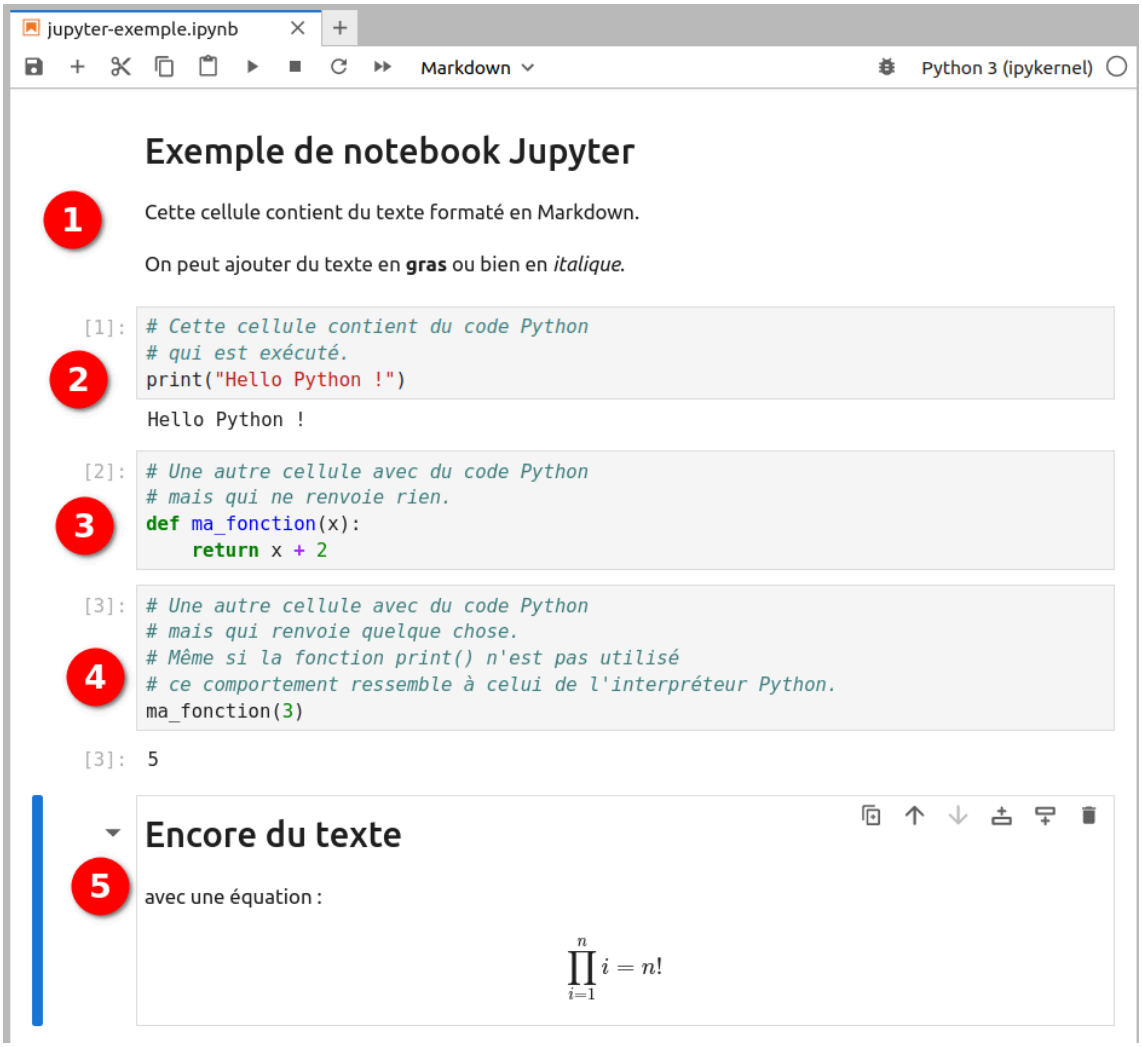


Figure 1: Illustration of a Jupyter notebook concept. Source: Python Course for Life Sciences (Paris 18) by Patrick Fuchs and Pierre Poulain

Initially, `Jupyter` was the only software offering these interactive features. There are now other ways to benefit from the advantages of notebooks while having an IDE with more complete features than `Jupyter`. This is why, as of 2026, it is more practical to use `VSCode`—a general-purpose code editor that nonetheless offers excellent `Python` features—rather than `Jupyter` directly. Already very handy for working with notebooks, a real IDE like `VSCode` will become essential once you start using `Python` through scripts.

> **i** Going further
>
> For more information on using notebooks in VSCode, refer to the official documentation.
>
> https://code.visualstudio.com/assets/docs/datascience/jupyter/notebook-run-in-section.mp4

3 Using notebooks in the context of this course

The best way to learn is through practice. To enable this, all chapters of this course will be executable and editable as *notebooks*. The following buttons allow you to open this chapter as a *notebook* in different environments:

1. On `Github` , only for viewing and downloading notebooks as `Github` is not a development and execution environment for notebooks;
2. On `SSPCloud`, a modern cloud platform developed by Insee and provided free of charge to public agents, students, researchers, and agents of European public statistical institutes. As mentioned in the dedicated box, this is the recommended entry point for notebooks for anyone who has access to it. Notebooks can be opened there via `VSCode` and come with command-line access and appropriate rights for package installation. In addition to these already desirable features for discovering `Python`, other useful features for continuous learning are explored in more detail in the third year in the course on "Deploying Data Science Projects": free GPU access, interfacing with other cloud technologies such as object storage systems, etc.
3. Google Colab is a free online service based on the `Jupyter` interface that provides access to `Python` resources run on Google's servers.
4. Github codespaces also allows opening the *notebooks*, but this option has not been tested for this course: the outcome is therefore not guaranteed.

> ! Recommended environment for this course
>
> For public sector employees or students from partner schools, it is recommended to use the `SSPCloud` button, which is a modern, powerful, and flexible cloud infrastructure developed by Insee and accessible at https://datalab.sspcloud.fr[6].

4 Exercise to Explore Basic Notebook Features

All the chapters of this course are designed as a narrative aiming to address a practical need with intermediate exercises serving as milestones. They are easily identifiable in the following format:

> Tip
>
> An example of an exercise box

The following exercise aims to familiarize you with using *notebooks* if you are not familiar with this type of document. It illustrates basic features such as writing code, executing cells, adding text, and visualizing data.

To do this, open this chapter in a notebook-compatible environment:[7]

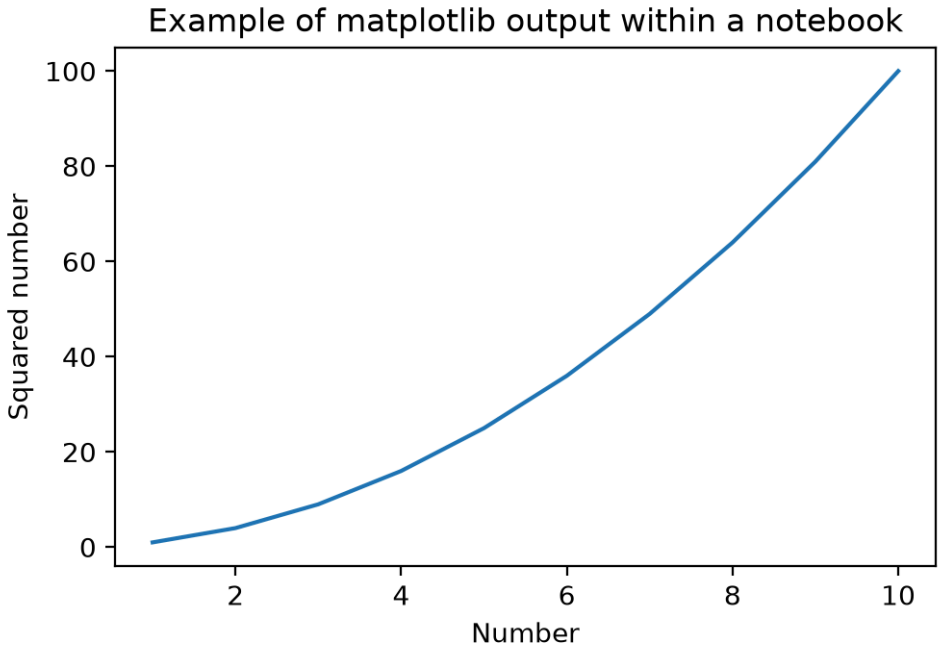
[6] For users of this infrastructure, the notebooks for this course are also listed, along with many other high-quality resources, on the Training page.

[7] To use https://datalab.sspcloud.fr if you are eligible, you need to create an account.

Exercise 1

1. Under this exercise, create a code cell. Write a `Python` code that displays the phrase: *"Welcome to a notebook!"* and then execute the cell. Modify your code and re-run it.
2. By searching online, add a new cell and change its type to "Markdown". In this cell, write a short text including the following elements:
   - A small part in italics
   - An unordered list
   - A level 2 heading (equivalent to `<h2>` in HTML or `\subsection` in $\LaTeX$)
   - An equation
3. Create a code cell anywhere in the document. Create a list of integers from 1 to 10 named `numbers`. Display this list.
4. Create a new code cell. Use the code below this exercise to generate a figure.

The figure obtained at the end of the exercise will look like this:



5 How to solve errors?

Encountering errors is completely natural and expected when learning (and even after!) a programming language. Resolving these errors is a great opportunity to understand how the language works and to become self-sufficient in its use.

Here is a suggested sequence of steps to follow (in this order) to resolve an error:

1. **Read the *logs* carefully**, i.e., the outputs returned by `Python` in case of an error. Often, they are informative and may contain the answer directly.
2. **Search on the internet** (preferably in English and on `Google`). For example, providing the error name and part of the informative error message returned by `Python` usually helps to orient the search results towards what you are looking for.

3. Often, the **search will lead you to the Stackoverflow forum**, which is designed for this purpose. If you really can't find the answer to your problem, you can post on `Stackoverflow` by detailing the problem encountered so that forum users can reproduce it and find a solution.
4. **Official documentation** (of `Python` and various packages) is often a bit dry but generally exhaustive. It helps to understand how to use the various objects. For example, for functions: what they expect as input, parameters and their types, what they return as output, etc.
5. Code assistant AIs (`ChatGPT`, `Claude`) can be very helpful. By providing them with appropriate instructions and verifying the generated code to avoid hallucinations, you can save a lot of time with these tools. Be careful, however, not to become lazy by systematically jumping to step 5: you risk becoming captive to these tools without ever having learned to be autonomous.

Exercise 2: Learning by Making Mistakes

Fix the cell below so that it no longer produces an error

```
pd.DataFrame(x)
```

```
NameError: name 'pd' is not defined
␛[31m---------------------------------------------------------------------------␛[39m
␛[31mNameError␛[39m                                 Traceback (most recent call last)
␛[36mCell␛[39m␛[36m ␛[39m␛[32mIn[4]␛[39m␛[32m, line 1␛[39m
␛[32m----> ␛[39m␛[32m1␛[39m pd.DataFrame(x)

␛[31mNameError␛[39m: name 'pd' is not defined
```

6 How to continue learning after this course

6.1 Website content

This course is an introduction to data science with `Python`. Most of the content is designed for those who are new to the subject or wish to explore a specific topic in this field, such as cartography or *machine learning*.

However, this course also reflects my nearly decade-long experience 👴 with `Python` on various data sources, infrastructures, and problems: it is thus somewhat editorialized ("*opinionated*" as the Anglo-Saxons would say) to highlight certain expectations for data scientists and to help you avoid the same pitfalls I encountered in the past.

This course also provides content to go beyond the first few months of learning. Not all the content on this site is taught; some advanced sections and even chapters are intended for continuous learning and can be used several months after discovering this course.

pythonds.linogaliana.fr is continuously updated to reflect the evolving `Python` ecosystem. The notebooks will remain available beyond the teaching semester.

6.2 Technical monitoring

The rich and thriving `Python` ecosystem means you must stay attentive to its developments to avoid becoming outdated. While with monolithic languages like `SAS` or `Stata` one could rely solely on official documentation without ongoing technical monitoring, this is not feasible with `Python` or `R`. This course itself is continuously evolving, which is quite demanding 😅, to keep up with the ecosystem changes.

Social networks like `LinkedIn`, and content aggregators like `Medium` or `Towards Data Science`, offer posts of varying quality, but maintaining continuous technical monitoring on these platforms is not useless: over time, it can help identify new trends. The site `Real Python` generally provides very good posts, comprehensive and educational. `Github` can also be useful for technical monitoring: by looking at trending projects, you can see the trends that will emerge soon.

Regarding printed books, some are of very high quality. However, it is important to pay attention to their update dates: the rapid pace of some elements of the ecosystem can render them outdated very quickly. It is generally more useful to have a recent but non-exhaustive post than a complete but outdated book.

Useful newsletters

There are many well-crafted newsletters for regularly tracking developments in the data science ecosystem. For me, along with social media, they are the primary source of fresh information.

If you had to subscribe to only one newsletter, the most important to follow is Andrew Ng's, *"The Batch"*. It provides reflections on academic advances in neural networks, evolution of the software and institutional ecosystem, making it an excellent *food for thought*.

The *newsletter by Christophe Bleffari*, aimed at data engineers but also of great interest to data scientists, often presents very good content. The *newsletter by Rami Krispin (data scientist at Apple)* is also very useful, especially when working regularly with both `Python` and and `Quarto`, the reproducible publishing software.

Fairly technical, the videos by Andrej Karpathy are very informative for understanding the workings of state-of-the-art language models. Similarly, content produced by Sebastian Raschka helps in knowing the latest advancements in research on the topic.

General newsletters from *Data Elixir* and Alpha Signal keep you updated with the latest news. In the field of data visualization, newsletters from *DataWrapper* provide accessible content on the subject.

Informations additionnelles

Python environment

This site was built automatically through a `Github` action using the `Quarto` reproducible publishing software (version 1.10.18).

The environment used to obtain the results is reproducible via uv. The `pyproject.toml` file used to build this environment is available on the `linogaliana/python-datascientist` repository

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
```

```
readme = "README.md"
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
    "scipy⩾1.13.0",
    "seaborn⩾0.13.2",
    "selenium<4.39.0",
    "spacy⩾3.8.4",
    "webdriver-manager⩾4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
```

```
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

To use exactly the same environment (version of `Python` and *packages*), please refer to the documentation for `uv`.